

## BAKI

### Analyse du fichier config/database.php de Baki

#### ✓ Les points forts

**Simplicité et efficacité** : Le code est concis et va droit au but. Pour un petit projet, cela fonctionne immédiatement sans complexité inutile.

**Gestion des erreurs** : L'utilisation du bloc try...catch montre que tu as conscience que la connexion à une base de données est une opération qui peut échouer.

**Mode d'exception** : Activer PDO::ERRMODE\_EXCEPTION est une bonne pratique pour ne pas laisser passer des erreurs SQL silencieuses.

#### Les points à surveiller

**Portée de la variable \$pdo** : Tu laisses \$pdo dans l'espace global. Cela peut poser des problèmes de "collision" si le projet s'agrandit ou si plusieurs fichiers sont inclus les uns dans les autres.

**Sécurité des messages d'erreur** : L'utilisation de echo "Erreur : " . \$e->getMessage(); est risquée. En cas de problème de configuration, cela pourrait afficher publiquement les identifiants ou le nom de la base de données.

**Paramétrage minimal** : Il manque la définition du charset (UTF-8). Sans cela, tu risques d'avoir des problèmes d'affichage avec les caractères spéciaux ou les accents dans les noms des contacts.

## Analyse du fichier models/Contact.php

### ✓ Les points forts

**Simplicité d'utilisation :** En utilisant des méthodes statiques (public static), tu permets au contrôleur d'utiliser le modèle sans avoir à l'instancier (ex: Contact::getAll()). C'est une approche rapide pour de petits utilitaires.

**Sécurité SQL :** Très bon point, tu utilises des requêtes préparées (prepare) avec des marqueurs de position (?). Même si tu utilises des index numériques au lieu de marqueurs nommés, la protection contre les injections SQL est bien présente.

**Concision :** Le code est extrêmement compact et facile à lire au premier coup d'œil.

### Les points à améliorer

**Utilisation de global \$pdo :** C'est le point le plus fragile. Dépendre d'une variable globale rend le code difficile à tester et à maintenir. Si la variable \$pdo est renommée dans la configuration, tout le modèle s'écroule.

**Manque de flexibilité :** Les méthodes statiques rendent l'**injection de dépendances** impossible. Si tu voulais un jour utiliser une base de données différente pour tes tests, tu ne pourrais pas facilement "injecter" une autre connexion dans sa classe.

**Gestion des données :** Tu ne précises pas le type des arguments (\$name, \$email, \$id). Cela laisse la porte ouverte à des erreurs de type (ex: passer un tableau au lieu d'une chaîne).

## Analyse de ContactController.php

### ✓ Tes points forts

**Rapidité d'exécution :** En appelant directement `Contact::getAll()` ou `Contact::add()`, tu évites d'instancier des objets inutilement. C'est une approche efficace pour une petite application de ce type.

**Validation des données :** Tu as bien pensé à nettoyer les entrées avec `trim()` et à vérifier la validité de l'email avec `filter_var`. C'est essentiel pour la propreté de ta base de données.

**Gestion des flux :** Tes redirections après chaque action (ajout ou suppression) sont bien placées. Cela évite les problèmes de renvoi de formulaire si l'utilisateur rafraîchit sa page.

### Tes points à améliorer

**Passage des messages d'erreur :** tu rediriges avec un simple paramètre dans l'URL (`?message=error`). C'est simple, mais l'utilisateur perd ce qu'il a saisi dans le formulaire s'il fait une erreur.

**Sécurité de la suppression :** Dans ta méthode `delete()`, tu récupères `$_GET['id']` sans vérifier s'il s'agit bien d'un nombre entier. Un utilisateur malveillant pourrait tenter de passer autre chose.

**Gestion des retours du Modèle :** Tu appelles `Contact::add()` sans vérifier si l'opération a réussi avant de rediriger avec le message `added`. Si la base de données est hors ligne, ton contrôleur dira quand même à l'utilisateur que tout s'est bien passé.

## Analyse de ton fichier views/contacts.php

### ✓ Tes points forts

**Esthétique Moderne :** Ton design est très réussi. L'utilisation des animations CSS (@keyframes fadeIn) et des ombres portées donne une impression de fluidité et de légèreté à l'application.

**Clarté du Feedback :** Tu as bien géré l'affichage des messages selon le paramètre reçu dans l'URL. L'utilisation de couleurs distinctes (vert pour le succès, rouge pour la suppression, orange pour l'erreur) permet à l'utilisateur de comprendre immédiatement ce qui se passe.

**Expérience Utilisateur (UX) :** Tu as inclus la confirmation de suppression en JavaScript (confirm), ce qui est une sécurité indispensable pour éviter les erreurs de clic.

### Tes points faibles (Attention !)

**Faible XSS critique :** C'est le point noir de ton projet. Tu affiches les données directement avec `<?= $contact['name'] ?>` sans utiliser `htmlspecialchars()`. Un utilisateur pourrait injecter du code JavaScript malveillant dans ta base de données (via le champ "Nom"), et ce code s'exécuterait chez tous les autres utilisateurs.

*Correction :* Tu devrais toujours écrire `<?= htmlspecialchars($contact['name']) ?>`.

**Logique dans la Vue :** Tu as inclus pas mal de logique PHP (if/elseif) pour traduire les messages directement dans le fichier HTML. Dans un MVC strict, on essaie souvent de préparer le texte du message dans le contrôleur pour que la vue n'ait qu'à l'afficher.

## Fiche d'Évaluation : Projet Répertoire MVC

Étudiant : Baki

Projet : Gestionnaire de Contacts (Style Modern Gradient)

Note Globale : 15 / 20

### Détail de l'Évaluation

| Critère               | Note    | Observations                                                                                                                                               |
|-----------------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Architecture MVC      | 3,5 / 5 | Le découpage est là, mais l'utilisation de méthodes statiques et de variables globales rend l'ensemble un peu rigide.                                      |
| Sécurité & Rigueur    | 2 / 5   | <b>Point d'attention :</b> L'absence de protection contre les failles XSS dans la vue pénalise l'ensemble. Les requêtes SQL sont cependant bien préparées. |
| Qualité du Code (PHP) | 4,5 / 5 | Ton code est extrêmement clair, concis et facile à lire. C'est ton point fort.                                                                             |
| Interface & UX (UI)   | 5 / 5   | Très beau travail sur le CSS. L'interface est l'une des plus agréables visuellement parmi les autres projets.                                              |